

```
# -----
# STEP 1: IMPORTING REQUIRED LIBRARIES
# -----
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

print("✓ Libraries successfully imported!")
```

✓ Libraries successfully imported!

```
# -----
# STEP 2: LOADING THE TELCO CHURN DATASET
# -----
print("--- LOADING DATASET ---")
url = "https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/master/data/Telco-Customer-Churn.csv"
df = pd.read_csv(url)

print(f"✓ Dataset loaded successfully!")
print(f"Initial Shape: {df.shape[0]} Rows and {df.shape[1]} Columns")
```

--- LOADING DATASET ---
 ✓ Dataset loaded successfully!
 Initial Shape: 7043 Rows and 21 Columns

```
# -----
# STEP 3: DATA CLEANING & QUALITY FIXES
# -----
print("--- IDENTIFYING AND FIXING DATA QUALITY ISSUES ---")

# Converting TotalCharges to numeric, setting invalid spaces to NaN
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')

# Imputing missing values with the median of the column
missing_before = df['TotalCharges'].isnull().sum()
df['TotalCharges'].fillna(df['TotalCharges'].median(), inplace=True)
missing_after = df['TotalCharges'].isnull().sum()

# Dropping unique identifier column
df_clean = df.drop(columns=['customerID'])

print(f"✓ Missing values in TotalCharges before: {missing_before}")
print(f"✓ Missing values in TotalCharges after imputation: {missing_after}")
print(f"✓ Unnecessary column 'customerID' dropped.")
print(f"Cleaned Dataset Shape: {df_clean.shape}")
```

--- IDENTIFYING AND FIXING DATA QUALITY ISSUES ---
 ✓ Missing values in TotalCharges before: 11
 ✓ Missing values in TotalCharges after imputation: 0
 ✓ Unnecessary column 'customerID' dropped.
 Cleaned Dataset Shape: (7043, 20)
 /tmp/ipykernel_2704/1974819483.py:11: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chainable .loc/.iloc. This behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting the values is a copy.
 For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value, inplace=True)

```
# -----
# STEP 4: EXPLORATORY DATA ANALYSIS (6 VISUALIZATIONS)
# -----
print("--- GENERATING 6 REQUIRED VISUALIZATIONS ---")
sns.set_theme(style="whitegrid")
plt.rcParams.update({'font.size': 10, 'axes.labelsize': 12, 'axes.titlesize': 14})

# Visualization 1: Target Variable Distribution
```

```
plt.figure(figsize=(5, 3.5))
sns.countplot(x='Churn', data=df_clean, palette='Set2')
plt.title('1. Distribution of Customer Churn (Target)')
plt.xlabel('Churn Status')
plt.ylabel('Count')
plt.tight_layout()
plt.show()

# Visualization 2: Tenure Distribution vs Churn
plt.figure(figsize=(8, 3.5))
sns.kdeplot(data=df_clean, x='tenure', hue='Churn', fill=True, common_norm=False, palette='crest', alpha=0.5)
plt.title('2. Customer Tenure Distribution by Churn')
plt.xlabel('Tenure (Months)')
plt.show()

# Visualization 3: Monthly Charges vs Churn
plt.figure(figsize=(8, 3.5))
sns.boxplot(x='Churn', y='MonthlyCharges', data=df_clean, palette='Set1')
plt.title('3. Monthly Charges vs Customer Churn')
plt.ylabel('Monthly Charges ($)')
plt.show()

# Visualization 4: Contract Type vs Churn
plt.figure(figsize=(7, 3.5))
sns.countplot(x='Contract', hue='Churn', data=df_clean, palette='viridis')
plt.title('4. Churn Rate by Contract Type')
plt.xlabel('Contract Type')
plt.show()

# Visualization 5: Internet Service Type vs Churn
plt.figure(figsize=(7, 3.5))
sns.countplot(x='InternetService', hue='Churn', data=df_clean, palette='magma')
plt.title('5. Churn Rate by Internet Service Type')
plt.xlabel('Internet Service Provider')
plt.show()

# Visualization 6: Correlation Matrix
plt.figure(figsize=(6, 4))
temp_df = df_clean.copy()
temp_df['Churn'] = LabelEncoder().fit_transform(temp_df['Churn'])
sns.heatmap(temp_df[['tenure', 'MonthlyCharges', 'TotalCharges', 'Churn']].corr(), annot=True, cmap='coolwarm', fmt=".2f", line
plt.title('6. Correlation Heatmap of Key Features')
plt.tight_layout()
plt.show()
```

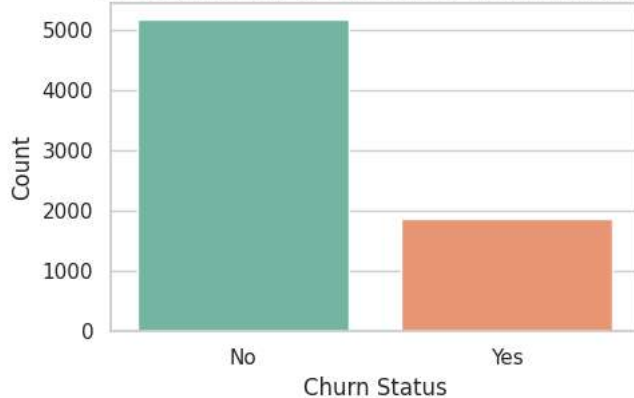

--- GENERATING 6 REQUIRED VISUALIZATIONS ---

/tmp/ipykernel_2704/829224337.py:10: FutureWarning:

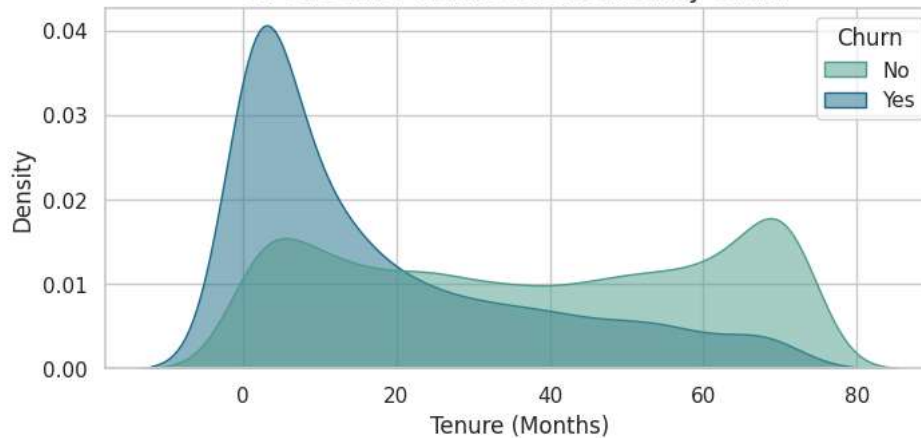
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set

```
sns.countplot(x='Churn', data=df_clean, palette='Set2')
```

1. Distribution of Customer Churn (Target)



2. Customer Tenure Distribution by Churn



/tmp/ipykernel_2704/829224337.py:26: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set

```
sns.boxplot(x='Churn', y='MonthlyCharges', data=df_clean, palette='Set1')
```

3. Monthly Charges vs Customer Churn

```
# -----
# STEP 5: PREPROCESSING PIPELINE (ENCODING & SCALING)
# -----
print("--- TRANSFORMATION PIPELINE INITIALIZED ---")

# Encoding categorical object text variables to numbers
le = LabelEncoder()
for col in df_clean.columns:
    if df_clean[col].dtype == 'object':
        df_clean[col] = le.fit_transform(df_clean[col])

# Separating Independent Features (X) and Target Label (y)
X = df_clean.drop(columns=['Churn'])
y = df_clean['Churn']

# Stratified 80/20 Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Feature Scaling using Standardisation
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f"✓ Encoding complete.")
print(f"✓ Feature scaling completed successfully.")
print(f"Training set size: {X_train.shape[0]} samples | Testing set size: {X_test.shape[0]} samples")
```

```
--- TRANSFORMATION PIPELINE INITIALIZED ---
✓ Encoding complete.
✓ Feature scaling completed successfully.
Training set size: 5634 samples | Testing set size: 1409 samples
```

```
# -----
# STEP 6: BASELINE & ADDITIONAL MODELS SETUP
# -----
print("--- MODEL DICTIONARY SETUP ---")

models = {
    "Logistic Regression (Baseline Model)": LogisticRegression(random_state=42),
    "Decision Tree (Additional Model 1)": DecisionTreeClassifier(max_depth=5, random_state=42),
    "Random Forest (Additional Model 2)": RandomForestClassifier(n_estimators=100, max_depth=5, random_state=42)
}

print(f"✓ Prepared models for training: {list(models.keys())}")
```

```
--- MODEL DICTIONARY SETUP ---
✓ Prepared models for training: ['Logistic Regression (Baseline Model)', 'Decision Tree (Additional Model 1)', 'Random Forest (Additional Model 2)']
```

```
# -----
# STEP 7: EVALUATION PIPELINE & REPORT GENERATION
# -----
print("--- RUNNING TRAINING & EVALUATION LOOP ---")

for name, model in models.items():
    # Fit the model on scaled training data
    model.fit(X_train_scaled, y_train)

    # Generate predictions on scaled test set
    y_pred = model.predict(X_test_scaled)

    print(f"\n===== {name} RESULTS =====")
    print(f"Final Model Accuracy: {accuracy_score(y_test, y_pred)*100:.2f}%")
    print("\nDetailed Classification Report:")
    print(classification_report(y_test, y_pred))
    print("Confusion Matrix Layout:")
    print(confusion_matrix(y_test, y_pred))
    print("=====\n")
```

```
--- RUNNING TRAINING & EVALUATION LOOP ---

===== Logistic Regression (Baseline Model) RESULTS =====
Final Model Accuracy: 79.91%

Detailed Classification Report:
      precision    recall  f1-score   support

      0       0.84      0.89      0.87      1035
      1       0.64      0.55      0.59       374

   accuracy          0.80      1409
  macro avg       0.74      0.72      0.73      1409
 weighted avg       0.79      0.80      0.79      1409

Confusion Matrix Layout:
[[921 114]
 [169 205]]
=====

===== Decision Tree (Additional Model 1) RESULTS =====
Final Model Accuracy: 78.50%

Detailed Classification Report:
      precision    recall  f1-score   support

      0       0.84      0.87      0.86      1035
      1       0.60      0.55      0.58       374

   accuracy          0.78      1409
  macro avg       0.72      0.71      0.72      1409
 weighted avg       0.78      0.78      0.78      1409

Confusion Matrix Layout:
[[899 136]
 [167 207]]
=====
```